JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ZoneOffsetTransition

java.lang.Object
    java.time.zone.ZoneOffsetTransition

All Implemented Interfaces:

Serializable, Comparable<ZoneOffsetTransition>

```
public final class ZoneOffsetTransition
extends Object
implements Comparable<ZoneOffsetTransition>, Serializable
```

A transition between two offsets caused by a discontinuity in the local time-line.

A transition between two offsets is normally the result of a daylight savings cutover. The discontinuity is normally a gap in spring and an overlap in autumn. `ZoneOffsetTransition` models the transition between the two offsets.

Gaps occur where there are local date-times that simply do not exist. An example would be when the offset changes from `+03:00` to `+04:00`. This might be described as 'the clocks will move forward one hour tonight at 1am'.

Overlaps occur where there are local date-times that exist twice. An example would be when the offset changes from `+04:00` to `+03:00`. This might be described as 'the clocks will move back one hour tonight at 2am'.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
int	**compareTo**(ZoneOffsetTransition ot	Compares this transition to another based on the transition instant.
boolean	**equals**(Object other)	Checks if this object equals another.

LocalDateTime	**getDateTimeAfter**()	Gets the local transition date-time, as would be expressed with the 'after' offset.
LocalDateTime	**getDateTimeBefore**()	Gets the local transition date-time, as would be expressed with the 'before' offset.
Duration	**getDuration**()	Gets the duration of the transition.
Instant	**getInstant**()	Gets the transition instant.
ZoneOffset	**getOffsetAfter**()	Gets the offset after the transition.
ZoneOffset	**getOffsetBefore**()	Gets the offset before the transition.
int	**hashCode**()	Returns a suitable hash code.
boolean	**isGap**()	Does this transition represent a gap in the local time-line.
boolean	**isOverlap**()	Does this transition represent an overlap in the local time-line.
boolean	**isValidOffset**(ZoneOffset offset)	Checks if the specified offset is valid during this transition.
static ZoneOffsetTransition	**of**(LocalDateTime transitio ZoneOffset offsetBefore, ZoneOffset offsetAfter)	Obtains an instance defining a transition between two offsets.
long	**toEpochSecond**()	Gets the transition instant as an epoch second.
String	**toString**()	

	Returns a string describing this object.

Methods declared in class Object

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Method Details

of

```
public static ZoneOffsetTransition of(LocalDateTime transition,
                                      ZoneOffset offsetBefore,
                                      ZoneOffset offsetAfter)
```

Obtains an instance defining a transition between two offsets.

Applications should normally obtain an instance from `ZoneRules`. This factory is only intended for use when creating `ZoneRules`.

Parameters:

`transition` - the transition date-time at the transition, which never actually occurs, expressed local to the before offset, not null

`offsetBefore` - the offset before the transition, not null

`offsetAfter` - the offset at and after the transition, not null

Returns:

the transition, not null

Throws:

`IllegalArgumentException` - if `offsetBefore` and `offsetAfter` are equal, or `transition.getNano()` returns non-zero value

getInstant

```
public Instant getInstant()
```

Gets the transition instant.

This is the instant of the discontinuity, which is defined as the first instant that the 'after' offset applies.

The methods `getInstant()`, `getDateTimeBefore()` and `getDateTimeAfter()` all represent the same instant.

Returns:

the transition instant, not null

toEpochSecond

```
public long toEpochSecond()
```

Gets the transition instant as an epoch second.

Returns:

the transition epoch second

getDateTimeBefore

```
public LocalDateTime getDateTimeBefore()
```

Gets the local transition date-time, as would be expressed with the 'before' offset.

This is the date-time where the discontinuity begins expressed with the 'before' offset. At this instant, the 'after' offset is actually used, therefore the combination of this date-time and the 'before' offset will never occur.

The combination of the 'before' date-time and offset represents the same instant as the 'after' date-time and offset.

Returns:

the transition date-time expressed with the before offset, not null

getDateTimeAfter

```
public LocalDateTime getDateTimeAfter()
```

Gets the local transition date-time, as would be expressed with the 'after' offset.

This is the first date-time after the discontinuity, when the new offset applies.

The combination of the 'before' date-time and offset represents the same instant as the 'after' date-time and offset.

Returns:

the transition date-time expressed with the after offset, not null

getOffsetBefore

```
public ZoneOffset getOffsetBefore()
```

Gets the offset before the transition.

This is the offset in use before the instant of the transition.

Returns:

the offset before the transition, not null

getOffsetAfter

```
public ZoneOffset getOffsetAfter()
```

Gets the offset after the transition.

This is the offset in use on and after the instant of the transition.

Returns:

the offset after the transition, not null

getDuration

```
public Duration getDuration()
```

Gets the duration of the transition.

In most cases, the transition duration is one hour, however this is not always the case. The duration will be positive for a gap and negative for an overlap. Time-zones are second-based, so the nanosecond part of the duration will be zero.

Returns:

the duration of the transition, positive for gaps, negative for overlaps

isGap

```
public boolean isGap()
```

Does this transition represent a gap in the local time-line.

Gaps occur where there are local date-times that simply do not exist. An example would be when the offset changes from `+01:00` to `+02:00`. This might be described as 'the clocks will move forward one hour tonight at 1am'.

Returns:

true if this transition is a gap, false if it is an overlap

isOverlap

```
public boolean isOverlap()
```

Does this transition represent an overlap in the local time-line.

Overlaps occur where there are local date-times that exist twice. An example would be when the offset changes from `+02:00` to `+01:00`. This might be described as 'the clocks will move back one hour tonight at 2am'.

Returns:

true if this transition is an overlap, false if it is a gap

isValidOffset

```
public boolean isValidOffset(ZoneOffset offset)
```

Checks if the specified offset is valid during this transition.

This checks to see if the given offset will be valid at some point in the transition. A gap will always return false. An overlap will return true if the offset is either the before or after offset.

Parameters:

`offset` - the offset to check, null returns false

Returns:

true if the offset is valid during the transition

compareTo

```
public int compareTo(ZoneOffsetTransition otherTransition)
```

Compares this transition to another based on the transition instant.

This compares the instants of each transition. The offsets are ignored, making this order inconsistent with equals.

Specified by:

`compareTo` in interface `Comparable`<`ZoneOffsetTransition`>

Parameters:

`otherTransition` - the transition to compare to, not null

Returns:

the comparator value, that is the comparison of this transition instant with `otherTransition` instant

equals

```
public boolean equals(Object other)
```

Checks if this object equals another.

The entire state of the object is compared.

Overrides:

`equals` in class `Object`

Parameters:

`other` - the other object to compare to, null returns false

Returns:

true if equal

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns a suitable hash code.

Overrides:

`hashCode` in class `Object`

Returns:

the hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
public String toString()
```

Returns a string describing this object.

Overrides:

`toString` in class `Object`

Returns:

a string for debugging, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.